Anti-Aliasing – Jittered Sampling

Mini Project – Report Part 1/2
Spring 2025

🧩Problem Definition :

Standard rendering with one ray per pixel causes aliasing artifacts, especially around edges and high-contrast transitions. This leads to jagged shapes, staircase effects, and visual distortion

💡Our Improvement:

We implemented Anti-Aliasing (AA) using Jittered Sampling:

Instead of casting a single ray through the center of each pixel, we cast multiple rays with random sub-pixel jittering. The colors of these rays are averaged to produce a smoother and .more accurate pixel color

🧱 Architectural Design :

Sample Rays Generation ✓

Implemented in method: constructRaysJ(int nX, int nY, int j, int i)

Rays are generated from the camera position through jittered locations inside the pixel square .area

Ray Casting ✓

.In renderImage(), we added a flag (AA_FLAG) to control whether to use AA

When AA is on: we use castRayJ(...) which collects multiple jittered rays, traces them, and .averages the result

.When AA is off: standard castRay(...) is used

Parameters ✓

Number of sample rays per pixel is configurable (samplesPerSide), typically 9 (3x3) or 16 (4x4), .and can be increased in tests

All constants and sampling logic are not hardcoded, allowing future extension (e.g., to .Stratified or Adaptive AA)

```java
public List<Ray> constructRaysJ(int nX, int nY, int j, int i) {  1 usage  NuritEzra

    List<Ray> rays = new ArrayList<>();
    Random random = new Random();
    double Ry = height / nY;
    double Rx = width / nX;
    double stepY = Ry / AA_GRID_SIZE;
    double stepX = Rx / AA_GRID_SIZE;

    for (int subI = 0; subI < AA_GRID_SIZE; subI++) {
        for (int subJ = 0; subJ < AA_GRID_SIZE; subJ++) {
            double jitterY = random.nextDouble() % AA_GRID_SIZE; // Random offset in Y direction
            double jitterX = random.nextDouble() % AA_GRID_SIZE; // Random offset in X direction

            double offsetI = (subI + jitterY) * stepY;
            double offsetJ = (subJ + jitterX) * stepX;
            double Yi = -(i - (nY - 1) / 2d) * Ry + offsetI;
            double Xj = (j - (nX - 1) / 2d) * Rx + offsetJ;
            Point pIJ = p0;
            if (!isZero(Xj)) pIJ = pIJ.add(vRight.scale(Xj));
            if (!isZero(Yi)) pIJ = pIJ.add(vUp.scale(Yi));
            pIJ = pIJ.add(vTo.scale(distance)); // pIJ is the center of the pixel in the view plane

            Ray ray = new Ray(p0, pIJ.subtract(p0).normalize());
            rays.add(ray);
        }
    }
    return rays;
}
```

```
*/
private void castRayJ(int nX, int nY, int j, int i) {  1 usage  Nuri

    List<Ray> rays = constructRaysJ(nX, nY, j, i);
    Color color = Color.BLACK;
    for (Ray ray : rays) {
        color = color.add(rayTracer.traceRay(ray));
    }
    color = color.scale( k: 1d / rays.size());
    imageWriter.writePixel(j, i, color);
```

🎯 Geometric Target Area :

Chosen target area: pixel square

.Sampling points are jittered inside a grid inside the pixel

.This ensures uniform and unbiased coverage across the pixel, avoiding visual distortion

🖼️ Output Example :

We created test scenes with:

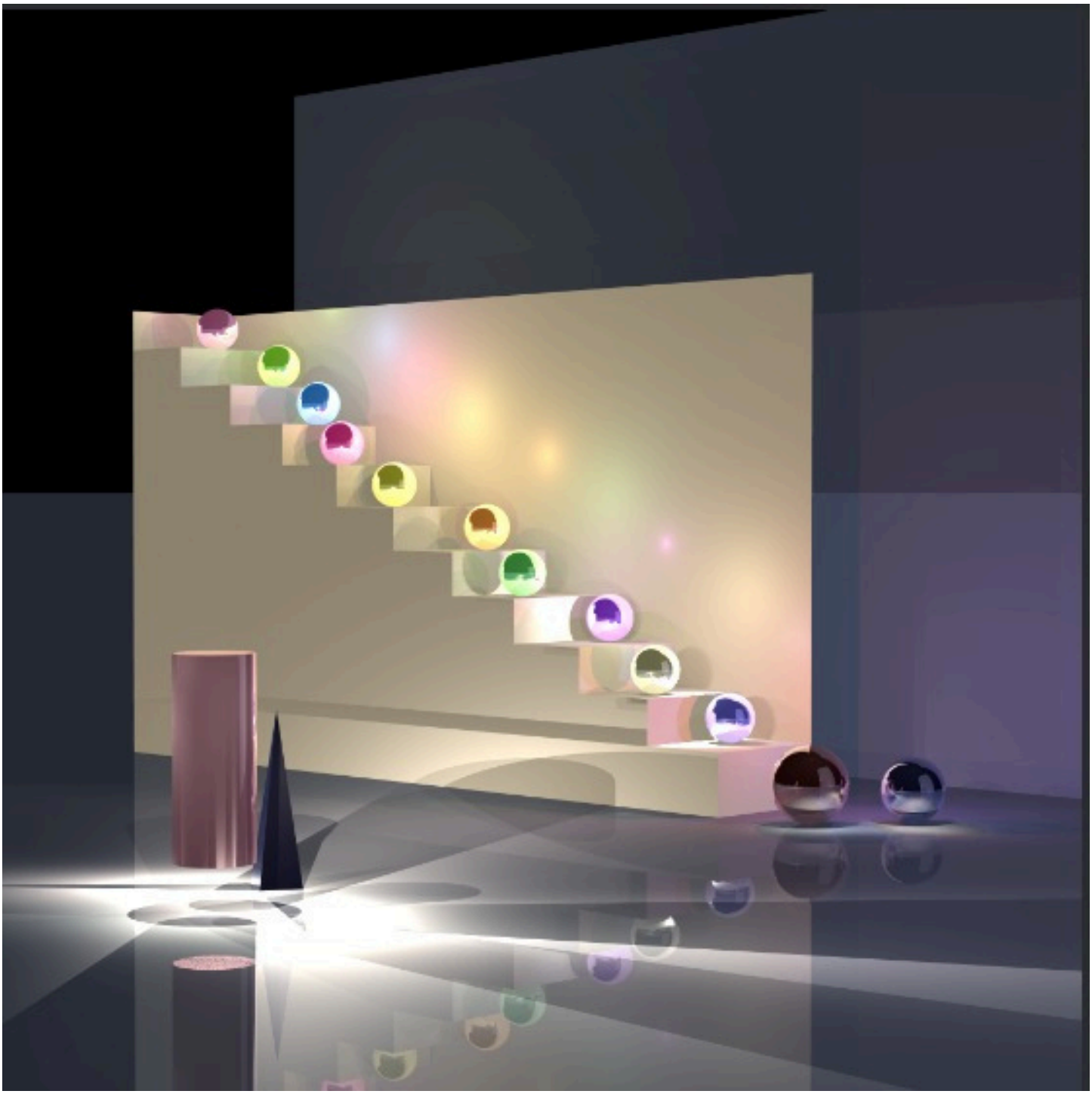
59 geometries.

51 light sources, each from different direction

One render with AA OFF, one with AA ON:

with AA OFF

with AA ON



🔁Difference in visual result

Edges are smoother

Contrast transitions more natural

Fine details are preserved better

:Performance trade-off 🕒

Rendering time increases linearly with number of rays

But the improvement in image quality justifies the cost

🧪Testability and Modularity:

Feature is fully modular and toggleable via flag (AA_FLAG) from test code

Can be tested in unit tests by comparing two render outputs (with/without AA)

Architecture allows reuse in other features like Depth of Field or Glossy Surface

⚙️Code Structure Summary :

renderImage(): Responsible for iterating over the image pixels and deciding whether to apply anti-aliasing (AA) based on the AA_FLAG. It delegates ray casting to either standard or jittered .methods

castRayJ(int nX, int nY, int j, int i): For each pixel (j,i), generates multiple jittered rays, traces .them, accumulates their colors, averages the result, and writes it to the image

constructRaysJ(int nX, int nY, int j, int i): Builds a list of rays with random (jittered) offsets .inside the pixel area to achieve more natural sampling and avoid grid patterns

rayTracer.traceRay(Ray ray): Traces a single ray in the scene and returns its color according to .scene geometry and lighting

The sampling logic is modular, making it easy to extend or replace the jittered AA technique .with other sampling methods (e.g., adaptive AA, stratified sampling)

✅Why It Works Well :

Jittered sampling is statistically unbiased

Eliminates structured patterns in sampling (like grid artifacts)

Compatible with future Monte Carlo or adaptive improvements

Mini Project – Report Part 2/2

Anti-Aliasing – Adaptive Sampling

🧩Problem Definition:

While jittered sampling improves image quality by using multiple rays per pixel, it samples uniformly across the image regardless of where more precision is required. This leads to .unnecessary computations in smooth regions without visual benefit

💡Our Improvement:

We implemented Adaptive Anti-Aliasing (Adaptive AA) to optimize sampling

.The pixel area is recursively subdivided

.Rays are cast at the corners of the pixel or sub-region

.If color variance between these rays is low → average and stop

.If variance is high → subdivide and continue sampling

This focuses computational effort on edges and areas of high detail, improving quality while .saving rays in flat regions

🧱Architectural Design:

✓Adaptive Ray Sampling

adaptiveCast

Implemented in method(...) :

This method recursively samples the pixel, computing variance and deciding whether to .subdivide further

.Casts rays at corners of a pixel or sub-region

.Computes color variance

.Decides whether to subdivide or average

✓Integration in renderImage()

.renderImage() checks AA_FLAG and invokes castRayAdaptive(...) if Adaptive AA is enabled

✓**Parameters**

.maxLevel: Maximum subdivision depth (e.g. 3-4)

.threshold: Color variance threshold to stop subdivision

Example of adaptiveCast code logic

```
private Color adaptiveCast(int nX, int nY, double minX, double minY, double maxX, double maxY,
                           int depth, int maxDepth, double threshold) {
    // Base case: maximum depth reached
    if (depth >= maxDepth) {
        Ray ray = constructRay(nX, nY, (minX + maxX) / 2, (minY + maxY) / 2);
        return rayTracer.traceRay(ray);
    }

    // Sample at corners and center of the region
    Ray rayTopLeft = constructRay(nX, nY, minX, minY);
    Ray rayTopRight = constructRay(nX, nY, maxX, minY);
    Ray rayBottomLeft = constructRay(nX, nY, minX, maxY);
    Ray rayBottomRight = constructRay(nX, nY, maxX, maxY);
    Ray rayCenter = constructRay(nX, nY, (minX + maxX) / 2, (minY + maxY) / 2);

    // Get colors for the five sample points
    Color colorTopLeft = rayTracer.traceRay(rayTopLeft);
    Color colorTopRight = rayTracer.traceRay(rayTopRight);
    Color colorBottomLeft = rayTracer.traceRay(rayBottomLeft);
    Color colorBottomRight = rayTracer.traceRay(rayBottomRight);
    Color colorCenter = rayTracer.traceRay(rayCenter);
```

```
Color averageColor = colorTopLeft.add(colorTopRight)
        .add(colorBottomLeft)
        .add(colorBottomRight)
        .add(colorCenter)
        .scale(0.2);

// Calculate the variance (measure of color differences)
double variance = calculateColorVariance(
        new Color[]{colorTopLeft, colorTopRight, colorBottomLeft, colorBottomRight, colorCenter},
        averageColor);

// If variance is low, we can stop subdividing
if (variance < threshold) {
    return averageColor;
}
```

```
        // Otherwise, recursively subdivide the region into four quadrants
        double midX = (minX + maxX) / 2;
        double midY = (minY + maxY) / 2;

        Color topLeftQuad = adaptiveCast(nX, nY, minX, minY, midX, midY,
                depth: depth + 1, maxDepth, threshold);
        Color topRightQuad = adaptiveCast(nX, nY, midX, minY, maxX, midY,
                depth: depth + 1, maxDepth, threshold);
        Color bottomLeftQuad = adaptiveCast(nX, nY, minX, midY, midX, maxY,
                depth: depth + 1, maxDepth, threshold);
        Color bottomRightQuad = adaptiveCast(nX, nY, midX, midY, maxX, maxY,
                depth: depth + 1, maxDepth, threshold);

        // Return the average color of the four quadrants
        return topLeftQuad.add(topRightQuad)
                .add(bottomLeftQuad)
                .add(bottomRightQuad)
                .scale( k: 0.25);
    }
```

Example of castRayAdaptive code

```
private void castRayAdaptive(int nX, int nY, int j, int i) {  3 usages  new *
    // Check if the coordinates are within the image bounds
    if (j < 0 || j >= Nx || i < 0 || i >= Ny) {
        return;  // Don't write if outside bounds
    }

    // Cast adaptive rays
    Color color = adaptiveCast(nX, nY, j, i,  maxX: j + 1,  maxY: i + 1,  depth
    // Write the result to the image
    imageWriter.writePixel(j, i, color);
    pixelManager.pixelDone();
}
```

🎯 Geometric Target Area:

Adaptive AA operates on pixel squares and dynamically subdivides based on the local color variation. This ensures efficient sampling where detail is needed without oversampling .smooth regions

🖼️ Output Example :

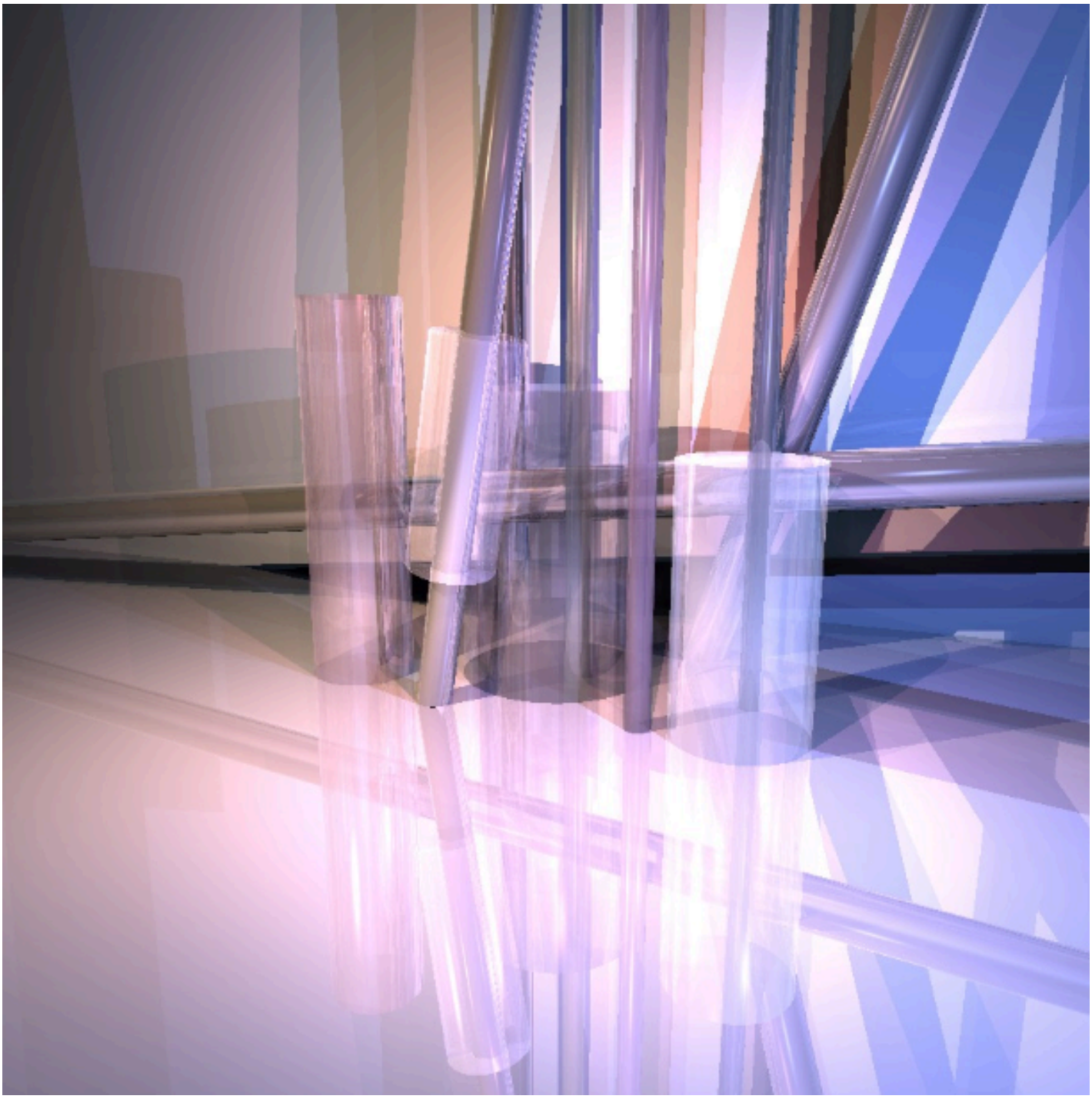
We created test scenes with:

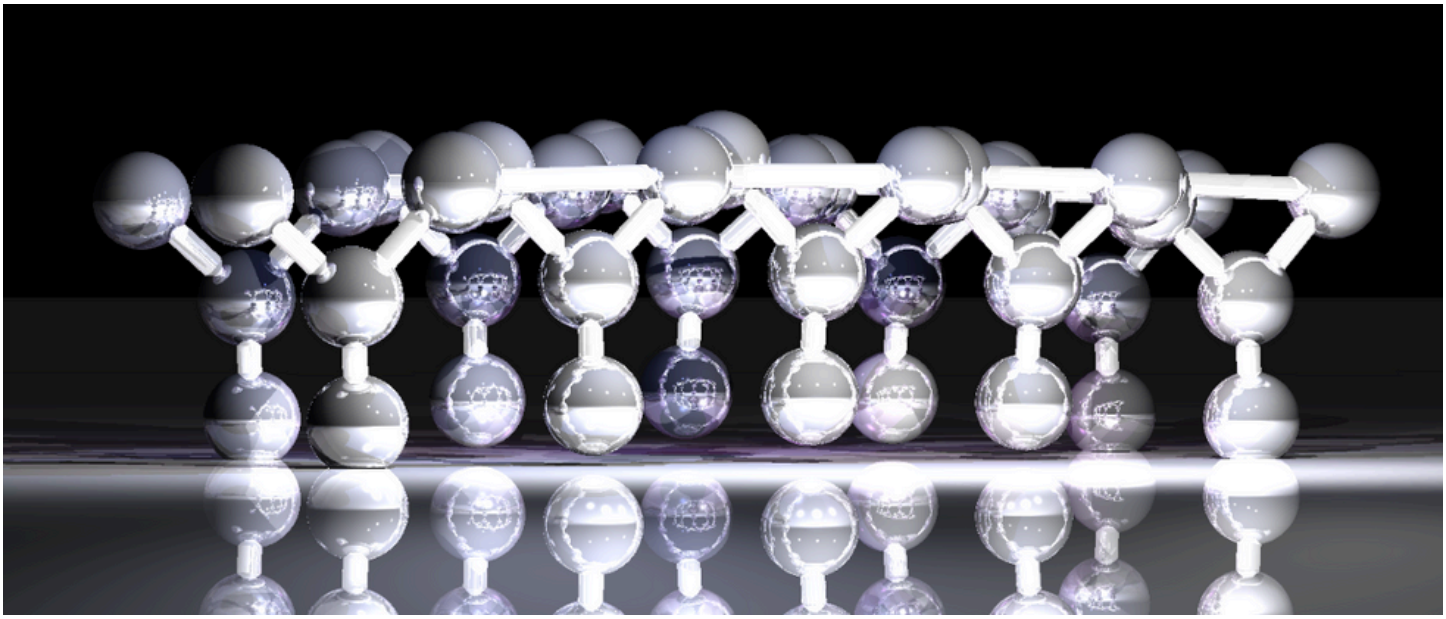
69 geometries.

7 light sources, each from different direction

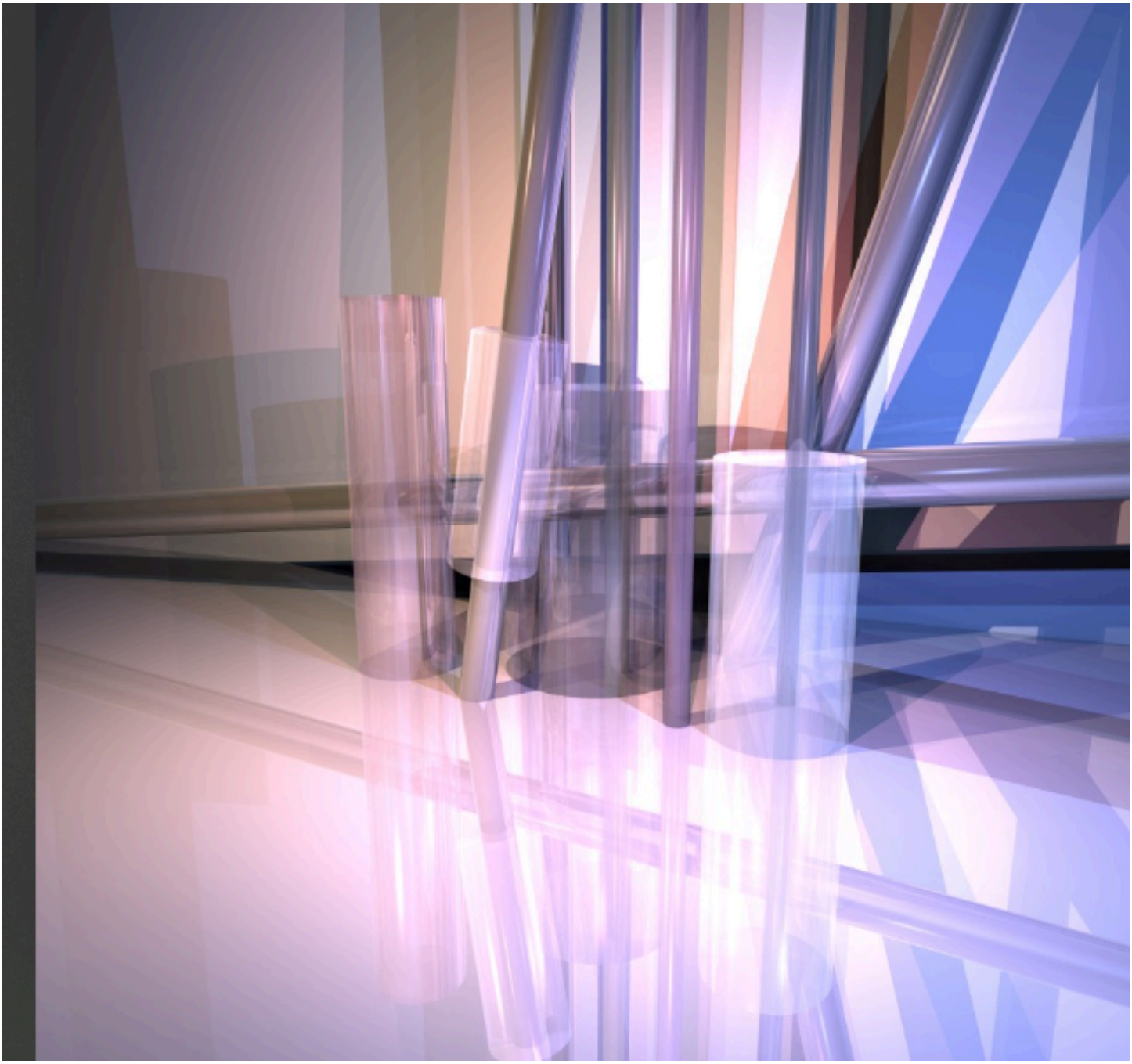
One render with AA OFF, one with AA ON:

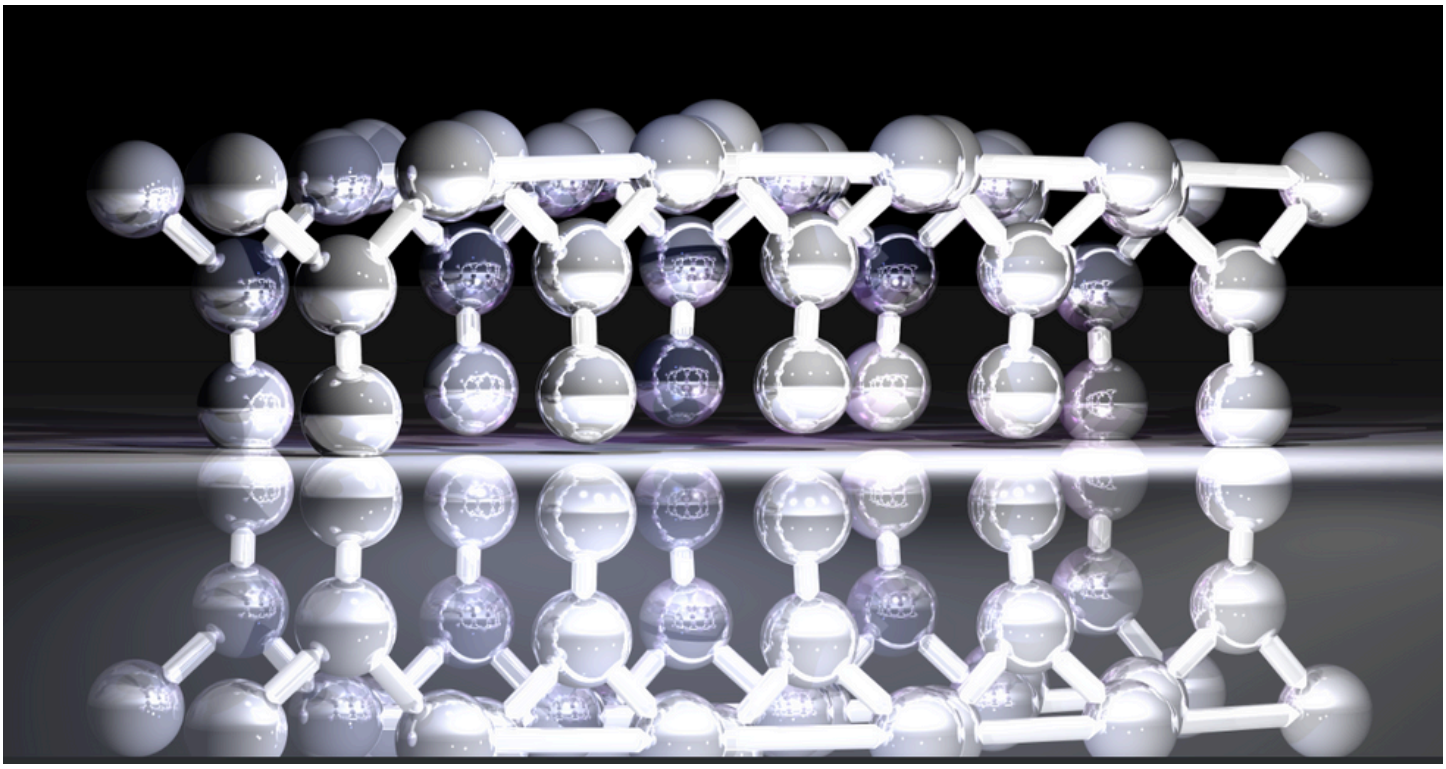
with AA OFF





with AA ON





Running time in the original image (above)

Without the Adaptive AA :90 min

With the Adaptive AA :10 min

🔁Difference in Visual Result:

.Smoother edges compared to no AA or uniform AA

.High-contrast transitions appear more natural

.Fewer rays are cast in uniform areas → better performance

🕒 Performance trade-off:

.Adaptive AA casts fewer rays overall than jittered AA

.It dynamically adjusts to image complexity → more efficient in most scenes

🧪Testability and Modularity:

.Adaptive AA is toggleable via ADAPTIVE_AA_FLAG

.Parameters (maxLevel, threshold) are adjustable for testing and tuning

.The code structure allows reuse in related features (e.g., soft shadows, glossy surfaces)

⚙️Code Structure Summary:

.castRayAdaptive(int nX, int nY, int j, int i): Initiates adaptive sampling for pixel (j,i)

.adaptiveCast(...): Recursive function that handles subdivision, variance check, averaging

.constructRay(...): Constructs rays through sub-pixel positions

.rayTracer.traceRay(Ray ray): Core ray tracing functionality for individual rays

✅Why It Works Well:

.Efficient: Focuses rays on regions that benefit most from detail

.Dynamic: Adjusts sampling to actual scene complexity

High quality: Produces better edge smoothness and detail preservation than uniform .sampling

Presented by:

Hadas Shor and Nurit Ezra

In life as in rendering, success comes from focusing effort where it makes the greatest impact, rather than .spreading yourself thin everywhere

בונוסים

נורמל לגליל סופי

חיתוך עם מצולע

חיתוך עם גליל אין סופי

חיתוך עם גליל סופי

חיתוך עם משולש

שימוש ב XML

סיום תוך שבוע את שלב 6